

第 22 章 计 算 图

22.1 引 言

在 AI 框架发展的最近一个阶段，技术上主要以计算图来描述深度神经网络。TensorFlow 和 PyTorch 代表了今天 AI 框架两种不同的设计路径：系统性能优先改善灵活性和灵活性易用性优先改善系统性能。这两种选择，随着神经网络算法研究和应用的进一步发展，使得 AI 框架在技术实现方案上出现了巨大的差异。图 22.1.1 是 AI 框架的示意图。

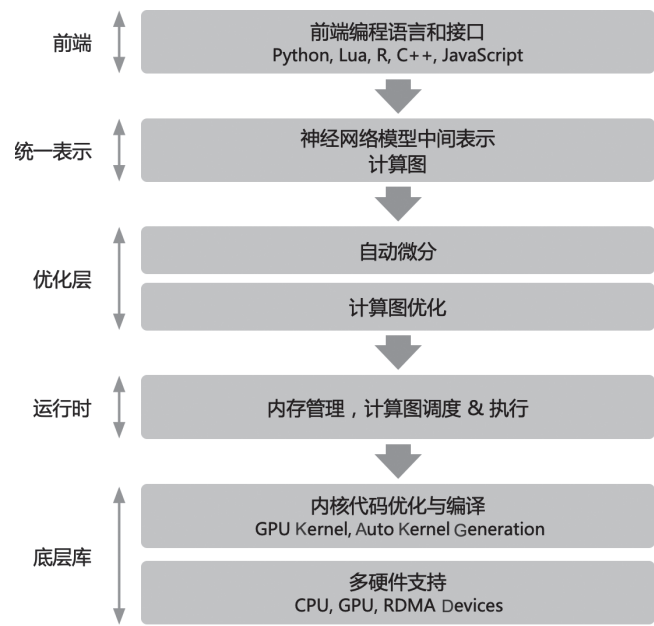


图 22.1.1

神经网络模型越来越复杂，包括混合专家模型（MOE）、生成对抗网络（GAN）、注意力模型（Attention Transformer）等。复杂的模型结构需要 AI 框架能够对模型算子的执行依赖关系、梯度计算以及训练参数进行快速高效的分析，便于优化模型结构、制定调度执行策略以及实现自动化梯度计算，从而提高 AI 框架训练的效率。

综上所述，目前主流的 AI 框架都选择使用**计算图**来抽象神经网络计算表达，通过通用的数据结构（张量）来理解、表达和执行神经网络模型，通过**计算图**可以把 AI 系统化的问题形象地表示出来。

神经网络的训练流程主要包括前向计算、计算损失、自动求导、反向传播和更新模型参数 5 个过程。在基于计算图的 AI 框架中，这 5 个阶段统一表示为由基本算子构成的计算图，算子是数据流图中的一个节点，由后端进行高效实现。

但是在程序实现过程中，会遇到很多编程性问题，例如控制流（if、else、while、for 等）与程序相关，而非与计算和数学表示的内容相关。因此会展开计算图和控制流之间的关系，也深入地去探讨计算关于调度和具体的执行方式，从而更好地、灵活地表达计算图。

22.2 计算图基本原理

本节将探讨 AI 概念落地时会遇到的一些问题与挑战，由此引出计算图的概念，以实现神经网络模型的统一抽象。接着展开讨论什么是计算以及计算图的基本构成，来深入了解计算图。最后简单地学习 PyTorch 如何表达计算图。

22.2.1 AI 系统化问题

1. 遇到的挑战

在真正的 AI 工程化过程中，我们会遇到诸多问题。为了高效地训练一个复杂神经网络，AI 框架需要解决这些问题，例如：

- 如何对复杂的神经网络模型实现自动微分？
- 如何利用编译期的分析 Pass 对神经网络的具体执行计算进行化简、合并、变换？
- 如何规划基本计算 Kernel 在计算加速硬件 GPU/TPU/NPU 上高效执行？
- 如何将基本处理单元派发（Dispatch）到特定的高效后端实现？
- 对通过神经网络的自动微分衍生的大量中间变量，如何进行内存预分配和管理？

为了使用统一的方式解决上述问题，AI 框架的开发者和架构师必须思考如何为各类神经网络模型的计算提供统一的描述，从而使得在运行神经网络计算之前，能够对整个计算过程尽可能进行推断，在编译期间自动为深度学习的应用程序补全反向计算、规划执行、降低运行时开销、复用和节省内存。能够更好地对领域特定语言（DSL），这里特指深度学习和神经网络，进行表示，并对使用 Python 编写的神经网络模型进行优化与执行。

因此，目前主流的 AI 框架都选择使用**计算图**来抽象神经网络计算。

2. 计算图的定义

有些 AI 框架把统一的图描述称为数据流图，有些称为计算图，本书统称为**计算图**。下面简

单介绍为什么可以都统称为计算图。

- **数据流图 (Data Flow Diagram, DFD)**：从数据传递和加工角度，以图形方式来表达系统的逻辑功能、数据在系统内部的逻辑流向和逻辑变换过程，是结构化系统分析方法的主要表达工具及用于表示软件模型的一种图示方法。在 AI 框架中数据流图表示对数据进行处理单元，接收一定的数据输入，然后对其进行处理，再进行系统输出。

- **计算图 (Computation Graph)**：被定义为有向图，其中节点对应于数学运算，计算图是表达和评估数学表达式的一种方式。而在 AI 框架中，计算图就是一个表示运算的有向无环图 (Directed Acyclic Graph, DAG)。

两者都把神经网络模型统一表示为图的形式，而图则是由节点和边组成。其都是描述数据在图中的节点传播的路径，是由固定的计算节点组合而成，数据在图中的传播过程，就是对数据进行加工计算的过程。

22.2.2 计算图的基本构成

1. 数据表达方式

1) 标量

标量 (Scalar)，亦称“无向量”。用通俗的说法，标量是只有大小，没有方向的量，如功、体积、温度等，与矢量相区别。物理学中，标量（或纯量）指在坐标变换下保持不变的物理量，即不论如何旋转或平移坐标系，标量的值都不会改变。

在 AI 框架或者计算机中，标量是一个独立存在的数，比如线性代数中的一个实数 488 就可以被看作一个标量，所以标量的运算相对简单，与平常做的算数运算类似。代码 `x` 则作为一个标量被赋值：

```
x = 488
```

2) 向量

向量 (Vector)是数学、物理学和工程科学等多个自然学科中的基本概念。指一个同时具有大小和方向，且满足平行四边形法则的几何对象。

在 AI 框架或者计算机中，向量指一系列顺序排列的元素，通常习惯用括号将这些元素括起来，其中每个元素都有一个索引值来唯一地确定其在向量中的位置。其有大小也有方向，以

$\mathbf{x}_{\text{vec}} = \begin{bmatrix} 1.1 \\ 2.2 \\ 3.3 \end{bmatrix}$ 为例，其代码 `x_vec` 则被作为一个向量被赋值：

```
x_vec = [1.1, 2.2, 3.3]
```

3) 矩阵

矩阵 (Matrix)是一个按照长方阵列排列的复数或实数集合，最早来自于方程组的系数及常

数所构成的方阵。这一概念由 19 世纪英国数学家凯利首先提出。矩阵是高等代数学中的常见工具，也常见于统计分析等应用数学学科中。

在机器学习领域经常被使用，比如有 N 个用户，每个用户有 M 个特征，那这个数据集就可以用一个 $N \times M$ 的矩阵表示，在卷积神经网络中输入模型的最初的数据是一个图片，读取图片上的像素点（Pixel）作为输入，一张尺寸大小为 256×256 的图片，实质上就可以用 256×256

的矩阵进行表示。以 $\mathbf{x}_{\text{mat}} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$ 为例，其代码 `x_mat` 则被表示为一个矩阵被赋值：

```
x_mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

图 22.2.1 对标量、向量、矩阵进行了形象化表示。



图 22.2.1

4) 张量

张量（Tensor）理论是数学的一个分支学科，在力学中有重要应用。张量这一术语起源于力学，它最初是用来表示弹性介质中各点应力状态的，后来张量理论发展成为力学和物理学的一个有力的数学工具。张量之所以重要，在于它可以满足一切物理定律必须与坐标系的选择无关的特性。张量概念是矢量概念的推广，矢量是一阶张量。

在几何代数中，张量是基于向量和矩阵的推广，通俗一点理解的话，可以将标量视为零阶张量，向量视为一阶张量，矩阵视为二阶张量。在 AI 框架中，所有数据将会使用张量进行表示，例如，图像任务通常将一幅图片组织成一个三维张量，张量的三个维度分别对应着图像的长、宽和通道数，一幅长和宽分别为 H, W 的彩色的图片可以表示为一个三维张量，形状为 (C, H, W) 。自然语言处理任务中，一个句子被组织成一个二维张量，张量的两个维度分别对应着词向量和句子的长度。

一组图像或者多个句子只需要为张量再增加一个批量（Batch）维度， N 张彩色图片组成的一批数据可以表示为一个四维张量，形状为 (N, C, H, W) 。

2. 张量和张量操作

在执行计算任务中，数据常常被组织成一个高维数组，整个计算任务的绝大部分时间都消

耗在高维数组上的数值计算操作上。高维数组和高维数组之上的数值计算是神经网络的核心，构成了计算图中最重要的一类基本算子。在 AI 框架的数据中主要有稠密张量和稀疏张量，这里先考虑最为常用的稠密张量。

张量作为高维数组，是对标量、向量、矩阵的推广。AI 框架对张量的表示主要有以下几个重要因素：

- **元素数据类型**：在一个张量中，所有元素具有相同的数据类型，如整型、浮点型、布尔型、字符型等数据类型格式。
- **形状**：张量每个维度具有固定的大小，其形状是一个整型数的元组，描述了一个张量的维度以及每个维度的长度。

例如，第 17 章 17.3 节图 17.3.1 是针对形状为 (3, 2, 5) 的三维张量进行表示。

虽然张量通常用索引来指代轴，但是始终要记住每个轴的含义。轴一般按照从全局到局部的顺序排序：首先是 Batch 轴，随后是空间维度，最后是每个位置的特征。这样，在内存中，特征向量就会位于连续的区域。例如，针对图 22.2.2 左边形状为 (3, 2, 4, 5) 的四维张量进行表示，其内存表示如图 22.2.2 右边所示。

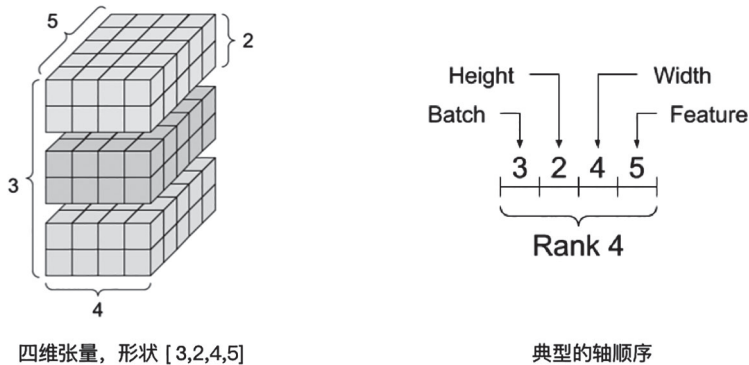


图 22.2.2

高维数组为开发者提供了一种逻辑上易于理解的方式，来组织具有规则形状的同质数据，极大地提高了编程的可理解性。另一方面，使用高维数组组织数据，易于让后端自动推断并完成元素逻辑存储空间向物理存储空间的映射。更重要的是：张量操作将同构的基本运算类型作为一个整体进行批量操作，通常都隐含着很高的数据并行性，因此非常适合在单指令多数据 (SIMD) 并行后端上进行加速。

3. 计算图表示 AI 框架

计算图是用来描述运算的有向无环图，有两个主要元素：节点 (Node) 和边 (Edge)。节点表示数据，如向量、矩阵、张量；边表示具体执行的运算，如加、减、乘、除和卷积等。

以简单的数学公式 $z = x + y$ 为例，可以绘制对应的计算图（图 22.2.3）。此计算图具有三个节点，分别代表张量数据中的两个输入变量 x 和 y 以及一个输出变量 z ，两条边带有具体的“+”符号表示加法。

在 AI 框架中会稍微有点不同，其计算图的基本组成有两个主要的元素：基本数据结构张量和基本计算单元算子。节点代表 Operator 具体的计算操作（即算子），边代表 Tensor 张量。整个计算图能够有效地表达神经网络模型的计算逻辑和状态。

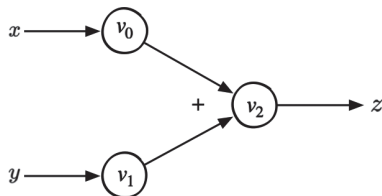


图 22.2.3

- **基本数据结构张量**：张量通过 Shape 来表示张量的具体形状，决定在内存中的元素大小和元素组成的具体形状；其元素类型决定了内存中每个元素所占用的字节数和实际的内存空间大小。
- **基本计算单元算子**：具体在加速器 GPU/NPU 中执行运算的是由最基本的代数算子组成，另外还会根据深度学习结构组成复杂算子。每个算子接受的输入输出不同，如 Conv 算子接受 3 个输入 Tensor，1 个输出 Tensor。

下面以简单的一个卷积、一个激活的神经网络模型的前向和反向为例，其前向的计算公式为

$$f(x) = \text{ReLU}(\text{Conv}(w, x, b))$$

反向计算微分的时候，需要加上损失函数：

$$\text{Loss}(x, x') = f(x) - x'$$

根据前向的神经网络模型定义，AI 框架中的计算图如图 22.2.4 所示。此计算图具有两个节点，分别代表卷积 Conv 计算和激活 ReLU 计算，Conv 计算接受三个输入：变量 x 、权重 w 和一个偏置项 b ，激活接受 Conv 卷积的输出并输出一个变量。图 22.2.4 中右图为对应左图的反向计算图，在神经网络模型训练的过程中，自动微分功能会为开发者自动构建反向图，然后输入输出一个完整 Step 计算。

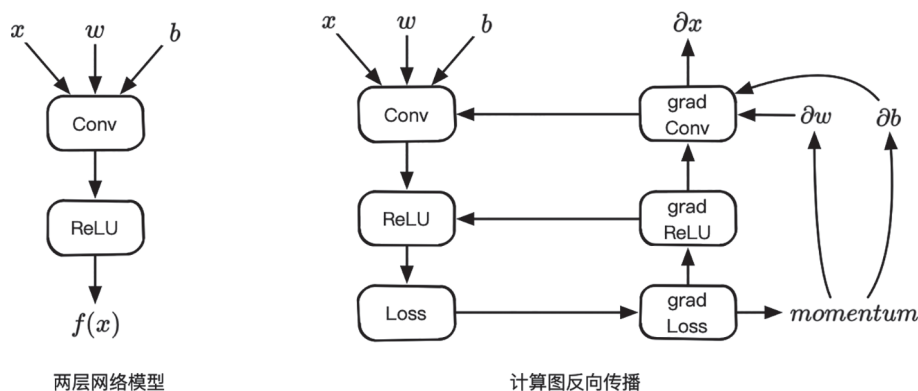


图 22.2.4

总而言之，AI 框架的设计很自然地沿用了张量和张量操作，将其作为构造复杂神经网络的基本描述单元，开发者可以在不感知复杂的框架后端实现细节的情况下，在 Python 脚本语言中复用由后端优化过的张量操作。而计算 Kernel 的开发者，能够隔离神经网络算法的细节，将张量计算作为一个独立的性能域，使用底层的编程模型和编程语言应用硬件相关优化。

这里的计算图其实忽略了 2 个细节：特殊的操作，如：程序代码中的 For/While 等构建控制流；特殊的边，如：控制边表示节点间依赖。

22.2.3 PyTorch 计算图

1. 动态计算图

在 PyTorch 的计算图中，同样由节点和边组成，节点表示张量或者函数，边表示张量和函数之间的依赖关系。其中 PyTorch 中的计算图是动态图。这里的动态主要有两重含义。

- 第一层含义是：计算图的前向传播是立即执行的。无需等待完整的计算图创建完毕，每条语句都会在计算图中动态添加节点和边，并立即执行前向传播得到计算结果。

```
import torch

w = torch.tensor([[3.0, 1.0]], requires_grad=True)
b = torch.tensor([3.0], requires_grad=True)
X = torch.randn(10, 2)
Y = torch.randn(10, 1)

# Y_hat 定义后其前向传播被立即执行，与其后面的loss创建语句无关
Y_hat = X@w.t() + b
print(Y_hat.data)

loss = torch.mean(torch.pow(Y_hat-Y, 2))
print(loss.data)
```

- 第二层含义是：计算图在反向传播后立即销毁。下次调用需要重新构建计算图。如果在程序中使用了 `Backward` 方法执行了反向传播，或者利用 `torch.autograd.grad` 方法计算了梯度，那么创建的计算图会被立即销毁，释放存储空间，下次调用需要重新创建。

```
# 如果再次执行反向传播将报错
loss.backward()

# 计算图在反向传播后立即销毁，如果需要保留计算图，需要设置 retain_graph = True
loss.backward(retain_graph = True)
```

2. 计算图中 Function

计算图中的另外一种节点是 `Function`，实际上为对张量操作的函数，其特点为同时包括前向计算逻辑和反向传播的逻辑。通过继承 `torch.autograd.Function` 来创建。

以创建一个 `ReLU` 函数为例：

```
class MyReLU(torch.autograd.Function):

    # 前向传播逻辑，可以用 ctx 存输入张量，供反向传播使用
    @staticmethod
    def forward(ctx, input):
        ctx.save_for_backward(input)
        return input.clamp(min=0)

    # 反向传播逻辑
    @staticmethod
    def backward(ctx, grad_output):
        input, = ctx.saved_tensors
        grad_input = grad_output.clone()
        grad_input[input < 0] = 0
        return grad_input
```

接着在构建动态计算图的时候，加入刚创建的 `Function` 节点。

```
# relu 现在也可以具有前向传播和反向传播功能
relu = MyReLU.apply
Y_hat = relu(X@w.t() + b)

loss = torch.mean(torch.pow(Y_hat-Y,2))
loss.backward()

print(w.grad)
print(b.grad)
print(Y_hat.grad_fn)
tensor([[4.5000, 4.5000]])
tensor([[4.5000]])
<torch.autograd.function.MyReLUBackward object at 0x1205a46c8>
```

小结与思考

- 计算图是 AI 框架中用于抽象和表达神经网络计算的关键数据结构，它将复杂的神经网络模型表示为一系列有向节点和边，便于自动微分和优化执行。

- 计算图由标量、向量、矩阵和张量等基本数据结构组成，并通过张量操作来执行数值计算，其中张量是多维数组的推广，适用于表达和处理数据并行性。
- PyTorch 中的计算图是动态的，前向传播即时执行，反向传播后计算图销毁，支持通过自定义 Function 来扩展新的操作，结合前向和反向逻辑实现自动微分。

22.3 计算图与自动微分

自动求导应用链式法则求某节点对其他节点的雅可比矩阵，它从结果节点开始，沿着计算路径向前追溯，逐节点计算。将神经网络和损失函数连接成一个计算图，则它的输入、输出和参数都是节点，可利用对损失函数进行求导，通过自动求导算法求得网络模型的雅可比矩阵，从而得到权重的梯度参数。

前面章节我们深入地探讨过自动微分的原理，可是 AI 框架中神经网络模型怎么跟自动微分产生连接关系呢？自动微分跟反向传播是什么关系呢？在实际 AI 框架的实现中，自动微分的实现方式有什么变化吗？

22.3.1 深度学习训练流程

1. 训练神经网络

深度学习的基本原理是设计一个多参数的非线性组合模型，即多层神经网络，因此深度学习的形式表示为神经网络模型。

通过神经网络模型可以用来近似（拟合）一个无法求解的复杂函数 $y = \text{fun}(x)$ ，其中输入变量 x 和输出变量 y 皆为高维变量。具体训练过程主要分为以下三个部分：

（1）**前向计算**：定义神经网络模型的前向传播过程，即网络训练的 Forward 部分，张量数据输入神经网络模型，模型输出具体的预测值，类似 $y = \text{fun}(x)$ 。这里的前向传播（Forward Propagation 或 Forward Pass）指的是按顺序（从输入层到输出层）计算和存储神经网络中每层的结果。

（2）**计算损失**：根据损失函数的定义，一般为真实样本的 (y, x) 和神经网络模型的预测 (y', z) 的比较函数。在损失函数中分别对每个维度的参数求其偏导数，得到每个参数的偏导数值即 `x_i.grad()`。

（3）**更新权重参数**：根据优化器（Optimizer）的学习策略，小幅通过反向计算图更新网络模型中的各个权重参数的梯度，即反向传播的过程（Backward Propagation 或 Backward Pass）。先看其梯度的 `grad` 正负，再根据正负方向对原参数值加减一定比例的梯度值。假设更新公式为 $w = w - n * \text{grad}$ ，如果梯度值为正，网络模型的权重参数就会减小；如果梯度值为负，网络模型的权重参数值就会增大。

在训练神经网络时，前向传播和反向传播相互依赖。前向传播沿着依赖的方向遍历计算图

并计算其路径上的所有变量。然后将这些用于反向传播，其中计算顺序与计算图相反。

以上述简单网络为例：一方面，在前向传播期间计算正则项取决于模型参数和的当前值。它们是由优化算法根据最近迭代的反向传播给出的。另一方面，反向传播期间参数的梯度计算，取决于由前向传播给出的隐藏变量的当前值。

在训练神经网络时，初始化模型参数后，我们交替使用前向传播和反向传播，利用反向传播给出的梯度来更新模型参数。

因此，深度学习神经网络训练的核心是求导，计算神经网络模型的参数 w ，并根据损失函数更新其梯度 $\frac{\partial L(w)}{\partial w}$ ，即

$$L(w) = \text{Loss}(f(w, x_i), y_i) \Rightarrow \frac{\partial L(w)}{\partial w}$$

2. 反向传播与自动微分

在计算图的概念中会经常提到自动微分功能，通过自动微分来构建反向的计算图。而在神经网络模型的训练流程和训练原理中，主要会提及反向传播算法，优化器对损失函数进行求导后的值，通过反向传播算法把传递给神经网络的每一层参数进行更新。那么在 AI 框架中，自动微分和反向传播之间的关系是什么呢？

首先，自动微分是将复合函数分解为输出变量（根节点）和一系列的输入变量（叶子节点）及基本函数（中间节点），构成一个计算图，并以此计算任意两个节点间的梯度：

- 加法法则：任意两个节点间的梯度为它们两节点之间所有路径的偏微分之和。
- 链式法则：一条路径的偏微分为路径上各相邻节点间偏微分的连乘。

而在神经网络中，只要各个组件以及损失函数都是可微的，那么损失函数就是关于各个输入变量的可微的复合函数。这个时候就可以使用自动微分的方式去计算神经网络模型里面的输入变量梯度，从而使用梯度下降算法减小误差。

因此，反向传播算法实际上就是自动微分，只不过在 AI 框架中，实际上计算图中的根节点为度量误差的损失函数，因而把节点间的偏导称为误差项。

22.3.2 自动微分基础回顾

神经网络模型的训练流程中，主要包含网络模型的前向计算、计算损失、更新权重参数三个计算阶段。

当开发者使用 AI 框架提供的 Python API 构造完成一个深度神经网络时，在数学上这个网络模型对应了一个复杂的带参数的高度非凸函数，求解其中的可学习参数依赖于基于一阶梯度的迭代更新法。

```
class LeNet(nn.Module):
    def __init__(self):
```

```

super(LeNet, self).__init__()
self.conv1 = nn.Conv2d(1, 6, 5, padding=2)
self.conv2 = nn.Conv2d(6, 16, 5)
self.fc1 = nn.Linear(16*5*5, 120)
self.fc2 = nn.Linear(120, 84)
self.fc3 = nn.Linear(84, 10)

def forward(self, x):
    x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
    x = F.max_pool2d(F.relu(self.conv2(x)), (2, 2))
    x = x.view(-1, self.num_flat_features(x))
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x

```

手工计算复杂函数的一阶梯度非常容易出错，自动微分（Automatic Differentiation, AD）就正是为了解决这一问题而设计的一种自动化方法。自动微分关注给定一个由原子操作构成的复杂前向计算程序，如何自动生成高效的反向计算程序。

自动微分按照工作模式可分为前向自动微分和反向自动微分，按照实现方式，自动微分又可为：

- 基于对偶数的前向微分；
- 基于 Tape 的反向微分；
- 基于源代码转换的反向微分。

1. 前向计算

自动微分是 AI 框架的核心组件之一，在讨论 AI 框架如何实现自动微分之前，我们先通过一个简单的例子理解自动微分的基本原理。

前向计算相关内容在 21.3.3 节已有详细介绍，现在简单回顾一下。假设一个简单的复合函数为例子：

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

该函数对应的计算图如图 21.3.2 所示。假设给定复合函数 $f(x_1, x_2)$ 中， x_1 和 x_2 均为输入变量。为了对 $f(x_1, x_2)$ 求值，依照表达式对应的计算图定义的计算顺序，复合函数 $f(x_1, x_2)$ 可以被分解成一个求值序列，把一个给定输入逐步计算输出的求值序列称为前向计算过程，参见图 21.3.3 左图。

前向计算过程是引入一系列的中间变量，将一个复杂的函数，分解成一系列基本函数或者叫作基本的计算操作，最后将这些基本函数构成一个前向的计算图。

2. 反向微分

链式求导法则是对称的，在计算导数 $\partial f(x_1, x_2) / \partial x$ 时，链式求导法则并不关心哪个变量作为分母，哪个变量作为分子。

反向微分相关内容在 21.3.4 节已有详尽介绍，总结而言，反向模式从后向前计算，依次得到每个中间变量节点的偏导数，直到到达自变量节点处，这样就得到了每个输入的偏导数。在每个节点处，根据该节点的后续节点（前向传播中的后续节点）计算其导数值。在反向微分中，变量导数的计算顺序与变量的前向计算顺序正好相反；运行的时间复杂度是 $O(m)$ ， m 是输出变量的个数。因此可以构建相对应的反向计算图，参见图 21.3.4 和图 21.3.5 右图。

在神经网络以及大量基于一阶导数方法进行训练的机器学习算法中，不论输入变量数目有多少，模型的输出一定是一个标量函数（即对应损失函数）。这决定了保留前向计算的所有中间结果，只需再次运行程序一次，便可以用反向微分算法计算出损失函数对每个中间变量和输入的导数。

3. 计算图的自动微分

在神经网络训练过程中，可以将每一个基本表达式理解为计算图中的一个节点，这个节点上面的标量形式的表达式进行了向量化的推广，因为 AI 框架的计算图中的数据统一表示为张量这一数据结构。

假设， $\mathbf{Y} = G(\mathbf{X})$ 作为基本求导原语，其中 $\mathbf{Y} = [y_1, \dots, y_m]$ 和 $\mathbf{X} = [x_1, \dots, x_n]$ 都是向量。这时， $\mathbf{Y}$ 对 $\mathbf{X}$ 的导数不再是一个标量，而是由偏导数构成的雅可比矩阵 $\mathbf{J}$ ：

$$\mathbf{J} = \left[\frac{\partial \mathbf{Y}}{\partial x_1}, \dots, \frac{\partial \mathbf{Y}}{\partial x_n} \right] = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}$$

反向传播算法过程中，即反向微分（自动微分的反向模式），中间层 $\mathbf{Y} = G(\mathbf{X})$ 会收到上一层计算出的损失函数对当前层输出的导数：

$$\mathbf{v} = \frac{\partial l}{\partial \mathbf{Y}} = \left[\frac{\partial l}{\partial y_1} \quad \dots \quad \frac{\partial l}{\partial y_m} \right]$$

然后将该导数继续乘以该层输出对输入的雅可比矩阵 $\mathbf{J}$ 向更上一层传播，这个乘法称为向量-雅可比乘积。

反向传播过程中如果直接存储雅可比矩阵，会消耗大量存储空间。但是，如果只存储向量-雅可比的乘积，在减少存储的同时并不会阻碍导数的计算。因此，AI 框架在实现自动微分时，对每个中间层存储的都是向量-雅可比的乘积，而非雅可比矩阵。

$$\mathbf{v} \cdot \mathbf{J} = \begin{bmatrix} \frac{\partial l}{\partial y_1} & \dots & \frac{\partial l}{\partial y_m} \end{bmatrix} \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_1}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial y_m}{\partial x_1} & \dots & \frac{\partial y_m}{\partial x_n} \end{bmatrix} = \begin{bmatrix} \frac{\partial l}{\partial x_1} & \dots & \frac{\partial l}{\partial x_n} \end{bmatrix}$$

AI 框架对于带有自动微分的计算图中的可导张量操作实现步骤具体如下：

- 同时注册前向计算节点和导数计算节点；
- 前向节点接受输入计算输出；
- 反向节点接受损失函数对当前张量操作输出的梯度 $\mathbf{v}$ ；
- 计算当前张量操作每个输入的向量-雅可比乘积。

在实际的 AI 框架构建的计算图中，并不是把前向节点和反向节点融合在一张图中，而是构建起计算图，如图 22.3.1 所示。

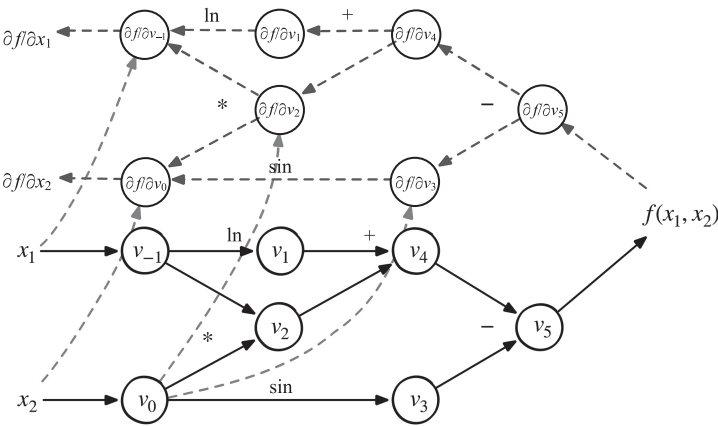


图 22.3.1

从图 22.3.1 中可知，前向计算图和反向计算图有着完全相同的结构，区别在于计算流动的方向相反。计算图中的每个节点都是一个无状态的张量操作，结点的入边（Incoming Edge）表示张量操作的输入，出边表示张量操作的输出。

同时，由于梯度会依赖前向计算的输入或计算结果，反向计算图中会有从前向计算图输入和输出张量指向反向计算图中导数计算节点的边。

22.3.3 AI 框架自动微分方式

在 AI 框架基于反向模式的自动求导机制中，根据反向计算图的构建时机，又分为基于对偶图的自动求导、基于表达式追踪和图层 IR 的自动求导，基于对偶图的求导方式通常与图层 IR

相结合；而使用动态图的 AI 框架多采用基于表达式追踪的自动求导方式（图 22.3.2）。

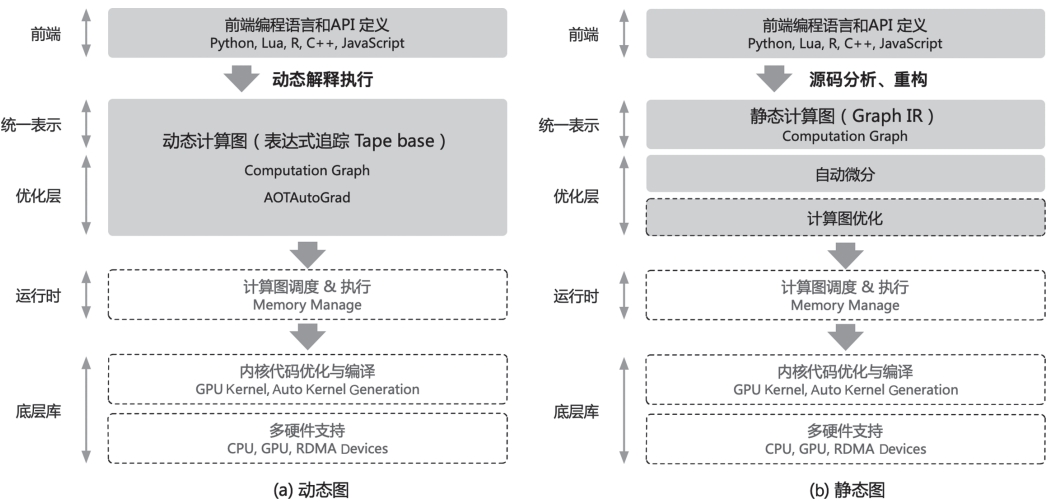


图 22.3.2

1. 动态计算图

AI 框架中实现自动微分最常用的一种方法是使用**表达式追踪（Evaluation Trace）**，即追踪数值计算过程的中间变量。简单地说，就是对应上一节说到的在前向计算过程中保留中间计算结果，根据反向模式的原理依次计算出中间导数，反向计算的过程中复用前向计算所保留的中间结果。

其中在业界主流的 AI 框架 PyTorch 和 PyTorch Autograd 都采用了一种称为**基于磁带（Tape-Based）**的自动微分技术：假设一个磁带式录音机来记录当前所执行的操作，然后它向后重放，来计算每一层的梯度。即会将上下文的变量操作都记录在 Tape 上，然后用反向微分法来计算这个函数的导数。

使用表达式追踪方式的优点在于方便跟踪和理解计算过程，易用性较高。缺点在于需要保存大量中间计算结果，内存占用会比静态图的实现方式要高。具体的实现细节上，Gradient Tape 默认只会记录对 Variable 的操作，主要原因是：

- Tape 需要记录前向传播的所有计算过程，之后才能计算反向传播；
- Tape 会记录所有的中间结果，不需要记录没用的操作。

与基于对偶图的自动求导机制的滞后性相反，前向传播过程中就可以构造出反向计算图，基于输出的梯度信息对输入自动求导。

2. 静态计算图

图层 IR 作为 AI 框架实现自动微分的另外一种方法。实现方式上，静态地生成可以根据 Python 等前端高级语言描述的神经网络拓扑结构，以及参数变量等图元信息，构建一个固定的计算图。这种实现方式也称为**静态计算图**。在基于计算图的 AI 框架中，利用反向微分计算梯度通常实现为计算图上的一个优化 Pass，给定前向计算图，以损失函数为根节点广度优先遍历前向计算图时，便能按照对偶结构自动生成反向计算图。

静态图在执行前有一个完整的构图和编译优化过程：

- 在构建前向图时，根据图元 IR 的定义，把 Python 等高级语言对神经网络模型的统一描述，通过源码转换成图元 IR 对应的前向计算图，将导数的计算也表示成计算图。
- 获取前向计算图后，根据自动微分的反向模式实现方法，执行前先生成反向对应的静态计算图，并完成对该计算图的编译优化，然后再给后端硬件执行具体的计算。

静态图的缺点在于，计算执行的过程中代码的错误不容易被发现，不能像动态图一样实时拿到中间的计算结果，给代码调试带来一定的麻烦。

好处在于，通过图元 IR 的抽象后，AI 框架已经构建起对计算图的统一描述，方便对全局的计算图进行编译优化，在执行期间可以不依赖前端语言描述，并且能够对图中的内存进行大量复用。因此常用于神经网络模型部署，如移动端安防领域、人脸识别等场景。

小结与思考

- 自动微分是 AI 框架实现神经网络训练的关键技术，它通过计算图来表达和自动求导，可以高效地计算损失函数对网络参数的梯度。
- 计算图将神经网络的前向计算过程表达为一系列节点和边，其中节点表示计算操作，边表示数据流，自动微分利用这些计算图来实现反向传播算法。
- AI 框架中的自动微分通常采用动态计算图或静态计算图的方式，动态图易于调试和支持灵活的神经网络结构调整，而静态图在执行效率和内存优化方面表现更优。

22.4 计算图的调度与执行

AI 框架在训练的过程中，会把神经网络统一表示为由基础算子构成的计算图，而算子属于计算图中的一个节点，由具体的后端硬件进行高效执行。

目前 AI 框架的前端负责给开发者提供对应的 API，通过统一表示把开发者编写的 Python 代码表示为前向计算图，AI 框架会根据前向计算图，自动补全反向计算图，生成出完整的计算图。

实际上，计算图的执行方式，可以分为两种模式：①逐算子下发执行的交互式方式，如 PyTorch 框架；②整个计算图或者部分子图一次性下发到硬件进行执行，如 TensorFlow 和

MindSpore。无论采用哪种模式，其大致架构如图 22.4.1 所示。

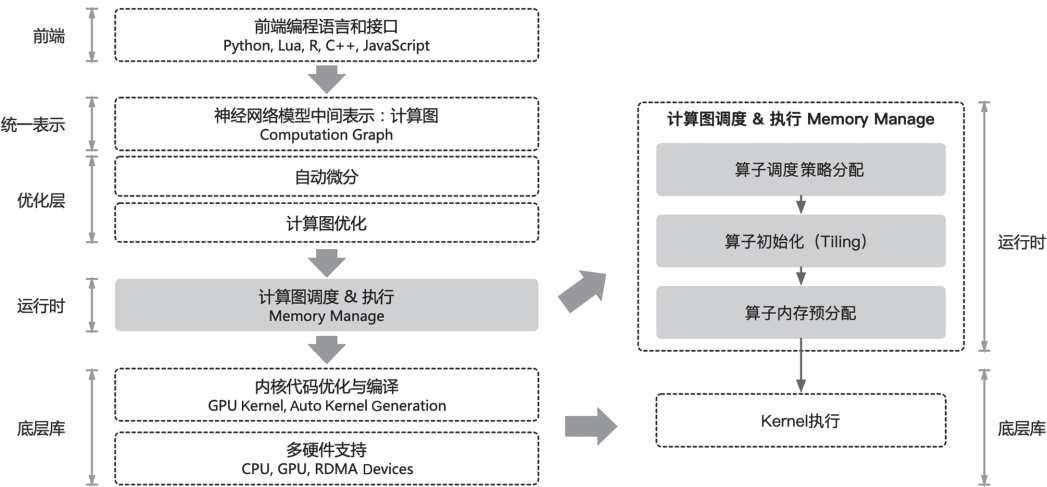


图 22.4.1

22.4.1 图调度

计算图的调度主要是指静态图。在静态图中，需要先定义好整个计算流，再次运行时就不需要重新构建计算图，因此其性能更加高效。

1. 什么是算子

AI 框架中对张量计算的种类有很多，比如加法、乘法、矩阵相乘、矩阵转置等，这些计算被称为算子（Operator），它们是 AI 框架的核心组件。算子的定义在第 1 章 1.4.1 节已介绍过。为了更加方便地描述计算图中的算子，现在来对**算子**这一概念的定义再次重复：

- **狭义的算子（Kernel）**：对张量执行的基本操作集合，包括四则运算、数学函数，甚至是对张量元数据的修改，如维度压缩（Squeeze）、维度修改（Reshape）等。

- **广义的算子（Function）**：AI 框架中对算子模块的具体实现，涉及调度模块、Kernel 模块、求导模块以及代码自动生成模块。

将狭义的算子统一称为核（Kernel），在 AI 框架中，使用 C++实现层里的算子指的就是这里的 Kernel，而这里的 Kernel 实现并不支持自动梯度计算（Autograd）模块，也不感知微分的概念。

将广义的算子称为函数或方法（Function），这也是平时经常接触到的 AI 框架中 PyTorch API，包括 Python API 和 C++ API，其配合 PyTorch Autograd 模块后就可以支持自动梯度求导计算。

2. 算子间调度

无论是大模型还是传统的神经网络模型，实际上最后执行都会落在单设备环境下执行对应的算子。对单设备执行环境，制约计算图中节点调度执行的关键因素是节点之间的数据流依赖和具体的算子。

假设继续以简单的复合函数为例子：

$$f(x_1, x_2) = \ln(x_1) + x_1x_2 - \sin(x_2)$$

图 22.4.2 是函数对应的计算图，一共有 5 个算子。

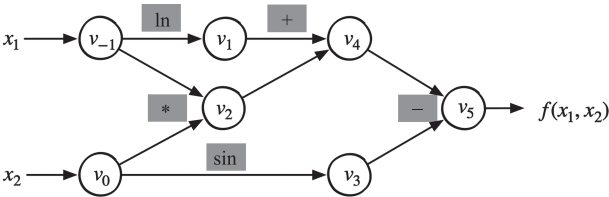


图 22.4.2

AI 框架根据上述计算图的数据流的依赖关系，在单设备环境下，依次调用具体的算子可以如下所示：

```
# 前向执行算子
Log(v_{-1}, 2) -> v1
Mul(v_{-1}, v0) -> v2
Sin(v0) -> v3
Add(v1, v2) -> v4
Sub(v4, v3) -> v5

# 反向执行算子
Sub_grad(v5, v5_delta) -> v4_delta
...
```

由于计算图准确地描述了算子之间的依赖关系，运行时的调度策略可以变得十分直接。根据计算图中的数据流依赖关系和计算节点函数，通过先进先出队列来执行具体的计算逻辑：

- 初始状态下，AI 框架会在运行时将计算图中入度为 0 的节点加入到 FIFO (First-In-First-Out) 队列中。
- 从 FIFO 队列中选择下一个节点，分配给线程池中的一个线程执行计算；
- 当前节点执行结束后，会将其后继节点加入就绪队列，当前节点出队；
- AI 框架在运行时继续处理 FIFO 队列中的剩余节点，直到遍历完所有的节点，队列为空。

图 22.4.3 是按照数据流约束执行对应的计算图的一个可能调度序列。其中灰色为前向计算时用到的算子，黑色为反向计算时用到的算子。这种调度方式主要以 PyTorch 的默认执行方式：TensorFlow 的 Eager 模式，以及 MindSpore 的 PyNative 模式为主。

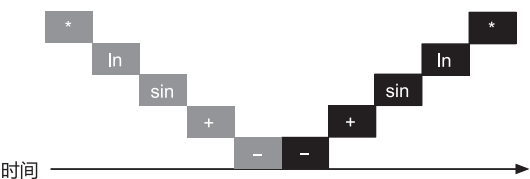


图 22.4.3

3. 算子并发调度

单设备算子间使用单线程管理先进先出队列进行调度，这种方式是最直接也是最原始的。实际 AI 框架会根据计算图，找到相互独立的算子进行并发调度，提高计算的并行性。

这个时候，就非常依赖于计算图能够准确地描述算子之间的依赖关系，通过后端编译优化功能或者后端编译优化的 Pass，提供并发执行队列的调度操作。

先进先出队列中的一个节点被分配给线程池中的线程调度执行时，这个线程会一次执行完计算图中所有低代价节点；部分 AI 框架会执行预编译阶段，在计算图调度模块中预先遍历计算图，区分高代价节点和低代价节点，并对其优先级根据具体情况进行等级划分。假设遇到高代价节点时，将该节点派发给线程池中其他线程执行，从而实现算子并发调度执行（图 22.4.4）。

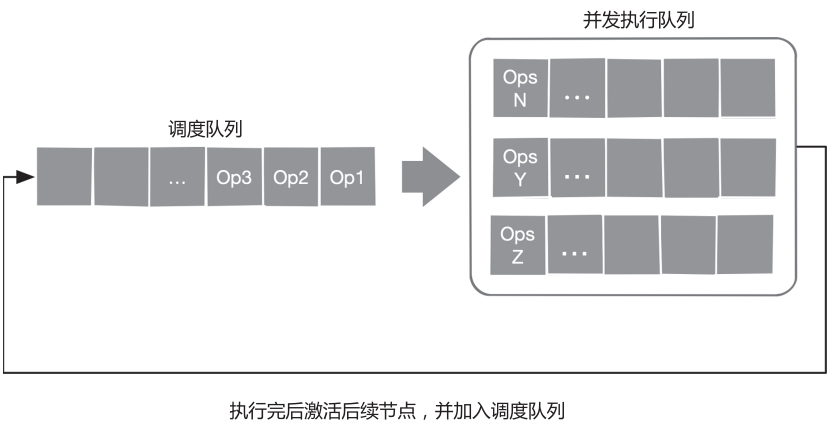


图 22.4.4

4. 算子异构调度

在手机端侧异构计算环境中，主要存在 CPU、GPU 以及 NPU 等多种异构的计算 IP，因此一张计算图可以由运行在不同计算 IP 的算子组成为异构计算图，继续以 $f(x_1, x_2) = \ln(x_1) +$

$x_1x_2 - \sin(x_2)$ 为例，图 22.4.5 展示了一个在端侧 SoC 中典型的由异构 IP 共同参与的计算图。

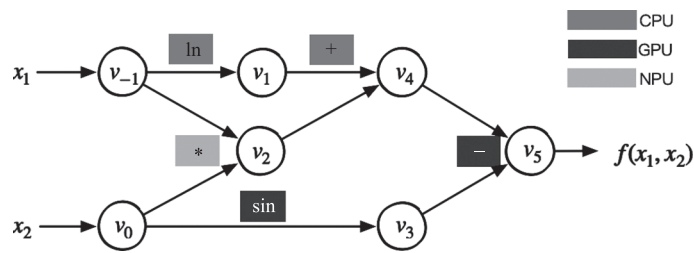


图 22.4.5

假设该手机 SoC 芯片有 CPU、GPU 和 NPU 三款计算 IP，所述计算图由如下几类异构计算 IP 对应的算子组成：

- CPU 算子：通过 CPU 执行的算子，CPU 计算的性能取决于是否能够充分利用 CPU 多核心的计算能力。
- GPU 算子：由 GPU 执行算子的计算逻辑，由于 GPU 具备并行执行能力，可以为高度并行的 Kernel 提供强大的并行加速能力。
- NPU 算子：由专门为高维张量提供独立 Kernel 计算的执行单元，NPU 优势是支持神经网络模型特殊的算子或者子图执行。

计算图能够被正确表达的首要条件是准确标识算子执行所在的不同设备，例如图 22.4.5 中，使用不同的灰度，标识 CPU、GPU 和 NPU Kernel，同一时间可以在不同的计算 IP 上执行不同的计算。目前主流 AI 框架均提供了指定算子所在运行设备的能力。

异构计算图的优点在：①异构硬件加速，将特定的计算放置到合适的硬件上执行；②算子间的并发执行，从计算图上可知，没有依赖关系的算子或者子图，逻辑上可以被 AI 框架并发调用。

不过从实际工程经验过程来看，目前采用算子异构调度的方式作为推理引擎的新增特性比较多，主要原因在于：①调度逻辑复杂，程序控制实现起来并不简单，即使自动化方式只能针对部分神经网络模型；②大部分神经网络模型的结构，仍然以高度串行为主，上下节点之间的依赖较重；③异构调度涉及不同 IP 计算结果之间的通信，通信的开销往往大于计算的开销。导致计算需要等待数据的同步与传输。

22.4.2 图执行

AI 框架生成计算图后，经过图调度模块对计算图进行标记，计算图已经准备好被实际的硬件执行，根据硬件能力的差异，可以将异构计算图的执行分为三种模式：单算子执行、整图下沉执行与图切分到多设备执行。

第一种单算子执行主要针对 CPU 和 GPU 的场景，计算图中的算子按照输入和输出的依赖关系被逐个调度与执行。整图下沉执行模式主要是针对 DSA 架构的 AI 芯片而言，其主要的优势是能够将整个计算图一次性下发到设备上，无需借助 CPU 的调度能力而独立完成计算图中所有算子的调度与执行，减少了主机和 AI 芯片的交互次数，借助 AI 芯片并行加速能力，提高计算效率和性能。由于计算图自身表达的灵活性，对于复杂场景的计算图在 AI 芯片上进行整图下沉执行的效率不一定能达到最优，或者在单个 AI 芯片上不能完整放下一张计算图。因此可以将计算图进行拆分，把大模型产生的计算图分别放在不同的 AI 加速芯片上面。此外，对于 AI 芯片执行效率低下的部分分离出来，交给 CPU 处理，将更适合 AI 芯片的子图下沉到 AI 芯片进行计算，这样可以兼顾性能和灵活性两方面。

1. 单算子执行

单算子执行类似于串行执行，将计算图展开为具体的执行序列，按照执行序列逐个 Kernel 执行，如图 22.4.6 所示。其特点为执行顺序固定，单线程执行，对系统资源要求相对较低。

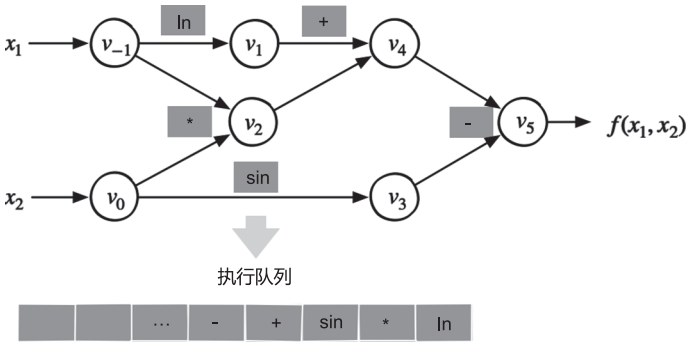


图 22.4.6

单算子执行的一般执行过程：算子在高级语言如 Python 侧被触发执行后，经过 AI 框架初始化，其中需要确定算子的输入输出数据、算子类型、算子大小以及对应的硬件设备等信息，接着 AI 框架会为该算子预分配计算所需的内存信息，最后交给具体的硬件加速芯片执行具体的计算。

单算子的执行方式好处在于其灵活性高，算子直接通过 Python 运行时调度：

- 通过高级语言代码表达复杂的计算逻辑，尤其是需要控制流以及需要高级语言的原生数据结构来实现复杂算法的场景；
- 便于程序进行调试，开发者可以在代码解释执行过程中控制需要调试的变量信息；
- 利用高级语言的特性，如在复杂计算加速任务中与 Python 庞大而丰富的生态库协同完成。

2. 整图下沉执行

单算子调度具有较高的易用性等优点，其缺点也很明显：

- 难于对计算图进行极致的性能优化，缺乏计算图的全局信息，单算子执行时无法根据上下文完成算子融合、代数简化等编译优化的工作；
- 缺乏计算图的拓扑关系，计算图在具体执行时退化成算子执行序列，只能按照给定的队列串行调度执行，即无法在运行时完成并行计算。

整图下沉式的执行方式，是通过专用的 AI 加速芯片，将整个计算图或者部分计算图（子图）一次性下发到 DSA 芯片上，以完成计算图的计算。如谷歌 TPU 和华为昇腾 NPU，多个算子可以组成一个子图，子图在执行之前被编程成一个具体的任务，将包含多个算子的任务一次性下发到硬件上直接执行。

计算图下沉的执行方式避免了在计算过程中，Host 主机侧和 Device 设备侧频繁地进行交互，CPU 下发一个算子到 NPU，再从队列中取出下一个节点下发到 NPU，因此可以获得更好的整体计算性能。然而计算图下沉执行的方式也存在一些局限，例如算子在动态 Shape、复杂控制流、副作用等场景下会面临较大的技术挑战。

22.4.3 PyTorch 算子执行

PyTorch 的函数是一个非常复杂核心的模块，其大部分代码都是由 PyTorch Tool 根据模板文件自动生成。如果想要查看其源代码，无法直接在 PyTorch 的 GitHub 代码库中搜索到，必须要将代码下载到本地并进行编译。当调用函数时，就会接触到 PyTorch 的调度模块。

以 PyTorch 的加法为例，假设调用 `torch.add` 函数 API 时，AI 框架总共会经历两次调度，如图 22.4.7 所示。

第一次调度会根据执行张量的设备（Device）和布局（Layout）动态选择对应的实现函数，比如 `<CPU, Strided> Tensor`，`<CPU, Sparse> Tensor` 或者 `<CUDA, Strided> Tensor`。不同设备布局的实现，可能会编译在不同的动态链接库里。

第二次调度则会根据张量元素的数据类型，通过 Switch 分支的方式进行一次轻量级的静态选择，最终选出合适的 Kernel 来执行对张量的操作。

Kernel 主要是算子的计算模块，但是别忘记了在深度学习中，算子还包含求导模块。计算模块主要定义了 Kernel 的计算步骤，需要先在 `aten/src/ATen/native/native_functions.yaml` 中声明 Kernel 计算模块的函数签名，然后在 `native/` 目录下实现该函数。

在前面的函数调用中，主要是通过 Kernel 对张量进行操作。求导模块主要是对计算模块的一个反向求导，需要直接在 `tools/autograd/derivatives.yaml` 中声明定义求导的过程，剩下就可以交给 Autograd 代码生成模块自动生成对应的代码。

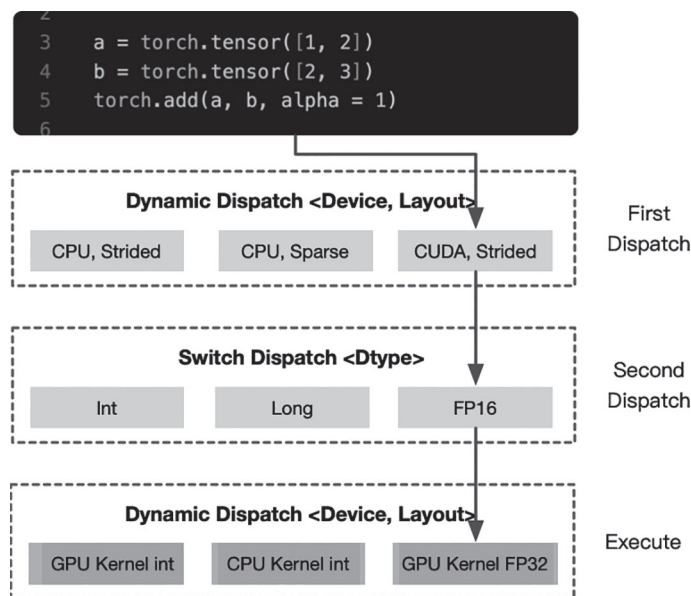


图 22.4.7

小结与思考

- 计算图的调度与执行是 AI 框架中的关键环节，涉及如何高效地管理和分配计算资源，以优化训练和推理过程。
- 调度主要分为图调度和算子间调度，包括静态图中的优化执行和单设备或异构计算环境中的并发及异构调度，以提高计算效率和资源利用率。
- 图执行方式包括单算子执行、整图下沉执行和图切分到多设备的并行执行，以适应不同的硬件架构和计算规模需求。
- PyTorch 中的算子执行涉及两次调度，首先是选择具体实现函数，然后是选择具体的 Kernel 执行操作，整个过程由 Autograd 模块支持自动求导。

22.5 计算图的控制流实现

计算图在数学上作为一个有向无环图（DAG），能够把神经网络模型的概念抽象出来作为统一描述，不过在计算机的编程中，会遇到很多 if、else、while、for 等控制流语句，有向无环图该如何表示控制流成了计算图其中一个很重要的问题。好处在于，引入控制流之后，开发者可以向计算图中引入分支选择以及循环控制逻辑，进而构造出更加复杂的神经网络模型结构。

目前，以 PyTorch 为例，它支持的仅仅是 Python Control Flow，即在 Python 层执行控制逻辑。

辑，而非计算图中支持控制流。这样就存在一个问题，如果要部署带 Control Flow 的模型就会比较困难，如何灵活部署带控制流的计算图到不支持 Python 的设备上？

计算图中的控制流实现与控制流图并不是一个概念。在计算机科学中，控制流图 (CFG) 是程序执行期间所有可能路径的图形表示。控制流图概念是由 Frances E. Allen 提出的。他指出，Reese T. Prosser 之前曾使用布尔连接矩阵进行流分析。CFG 是许多编译器优化和静态分析工具不可或缺的一部分。

22.5.1 控制流

AI 框架作为一个可编程系统，在设计时一个首要设计选择是，如何让开发者能够独立于实现细节，以最自然的方式描述出各类神经网络模型。描述的完备性不仅影响 AI 框架所能够支持的神经网络结构，决定了开发者在使用高级编程语言去实现神经网络模型的灵活性，也影响 AI 框架后端优化的技术手段。

1. 背景

在计算机科学中，控制流 (Control Flow) 定义了独立语句、指令、函数调用等执行或者求值的顺序。例如，根据函数 A 的输出值选择运行函数 B 或者 C 中的一个 (图 22.5.1)。

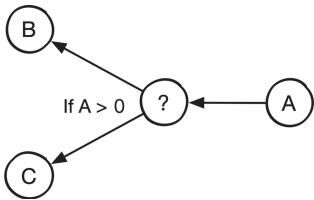


图 22.5.1

AI 框架把神经网络的计算过程，抽象为有向无环图。使用有向无环图描述神经网络计算的方式，符合算法开发者对神经网络概念的定义：算子间拓扑结构对学习特性有重要影响，可以通过计算图，方便地描述出大多数通过堆叠深度或多分支形成的复杂神经网络。然而，随着神经网络算法的快速发展，一些新颖的神经网络结构很难自然地表示为纯计算图。

2. 难点

引入控制流将会使计算图的构建以及前向传播存在很大的差异。
首先，计算图将变为动态的方式，分支选择以及循环控制流只有在真实运行的时候，才能够依据其依赖的数据输入来判断走哪个分支、是否结束循环。

其次，控制流引入的另一个难点在于循环控制流的实现。引入循环之后，原本的计算图在逻辑上出现了环，从而无法进行有效的拓扑排序。所以对于有控制流计算图，前向计算和反向传播的实现要么抛弃拓扑排序这一思路，要么就要通过其他手段将循环进行拆解。

为了能够支持含有控制流结构的神经网络模型，AI 框架需要引入对动态控制流这一语言结构（Language Construct）的支持。目前基于计算图，在控制流解决方案上，主要采用了三类设计思路：

- **复用宿主语言**：复用前端宿主语言的控制流语言结构，用前端语言中的控制逻辑驱动后端计算图的执行。
- **支持控制流原语**：AI 框架的后端对控制流语言结构进行原生支持，计算图中允许计算流和控制流混合。
- **源码解析**：前端对高级编程语言的代码解析成计算图，后端对控制流语言结构解析成子图，对计算图进行延伸。

复用宿主语言以 PyTorch 为典型代表，支持控制流原语以 TensorFlow 为典型代表，源码解析的方式则以 MindSpore 为典型代表。

22.5.2 动态图

每一次执行神经网络模型，AI 框架会依据前端编程语言描述，动态生成一份临时的计算图（实际为单算子），这意味着该实现方式下计算图是动态生成的，并且过程灵活可变，该特性有助于在神经网络结构调整阶段提高效率，这种实现方式也被称为**动态计算图**。

1. 复用宿主语言

PyTorch 采用的是动态图机制（Dynamic Computational Graph），动态图使得 PyTorch 的调试变得十分简单，每一个步骤，每一个流程都可以被开发者精确地控制、调试、输出，甚至是在每个迭代都能够重构整个网络。

动态图中，通过复用宿主语言的控制流构建动态图，即复用 Python 等高级语言本身的控制流的执行方式。以 PyTorch 为代表，在实际执行计算中，当遇到 if、else、for 等控制流语句时，使用 Python 在 CPU 的原控制执行方式，当遇到使用 PyTorch 表示神经网络的 API 时，才继续按照动态计算图的执行方式进行计算。

下面的代码是 Transformer 结构对应的动态图示例 PyTorch 代码，Transformer 的 Decoder 结构中通过 for 循环，堆叠 Decoder 模块，最后通过 if 来检查是否需要加入 norm 正则化层。此时 AI 框架不再维护一个全局的神经网络描述，神经网络变成具体的 Python 代码，后端的张量计算以库的形式提供，维持了与 Numpy 类似的一致编程接口。

```
from torch import nn

class TransformerDecoder(nn.Module):
    def __init__(self, decoder_layer, num_layers, norm=None):
```



```

super().__init__()
self.layers = _get_clones(decoder_layer, num_layers)
self.num_layers = num_layers
self.norm = norm

def forward(self, tgt: Tensor,
            memory: Tensor,
            tgt_mask: Optional[Tensor] = None,
            memory_mask: Optional[Tensor] = None,
            tgt_key_padding_mask: Optional[Tensor] = None,
            memory_key_padding_mask: Optional[Tensor] = None,
            tgt_is_causal: Optional[bool] = None,
            memory_is_causal: bool = False):
    seq_len = _get_seq_len(tgt, self.layers[0].self_attn.batch_first)
    tgt_is_causal = _detect_is_causal_mask(tgt_mask, tgt_is_causal, seq_len)

    for mod in self.layers:
        output = mod(output, memory, tgt_mask=tgt_mask,
                    memory_mask=memory_mask,
                    tgt_key_padding_mask=tgt_key_padding_mask,
                    memory_key_padding_mask=memory_key_padding_mask,
                    tgt_is_causal=tgt_is_causal,
                    memory_is_causal=memory_is_causal)

    if self.norm is not None:
        output = self.norm(output)

    return output

```

虽然 PyTorch 2.X 版本推出了图模式，能够生成真正的计算图，但是目前的方案中，遇到控制流，仍然会把网络模型切分成不同的子图来执行，遇到控制流会使用 Python 来执行调度。

2. 复用宿主语言特点

复用宿主语言的方式，其**优点**在于：

- 由于用户能够自由地使用前端宿主语言 Python 代码中的控制流，即时输出张量计算的求值结果，有着更高的易用性；
- **模型即代码**，动态图使用声明式编程的方式，使得定义神经网络模型的计算就像普通编写真正的程序。

这种复用宿主语言控制流语言驱动后端执行的方式有着更加友好的用户体验，但**缺点**也是明显的：

- 用户易于滥用前端语言特性，带来更复杂的性能问题；
- 计算图的执行流会在语言边界来回跳转，带来十分严重的运行开销；
- 整体计算图把模型切分成按算子组成的小块，不利于神经网络模型大规模部署和应用；
- 控制流和数据流被严格地隔离在前端语言和后端编译优化之中，前端和后端执行不同的语言和编译体系，跨语言边界优化难。

22.5.3 静态图

AI 框架静态地生成可以根据 Python 等前端高级语言描述的神经网络拓扑结构，以及参数变量等图层信息构建一个固定的计算图，这种实现方式也称为**静态计算图**。具体在 AI 框架中，利用反向微分计算梯度通常实现为计算图上的一个优化 Pass，给定前向计算图，以损失函数为根节点广度优先遍历前向计算图时，便能按照对偶结构自动生成反向计算图。

1. 控制流原语

支持控制流原语以 TensorFlow 为典型代表，在面对控制流需求时，计算图中引入控制流原语，运行时对控制流原语以第一等级（First-Class）进行实现支持。其控制流的基本设计原则是：引入包含少量操作的原子操作符（即 Function 类特殊算子），在这些操作符之上来表达 TensorFlow 应用的复杂控制流。

TensorFlow 计算图中支持控制流的方案主要分为 3 层。暴露给开发者用于构建计算图的前端 API，这些 API 会被转换成更低等级的控制流原语，再由计算图优化器进一步进行改写。为了平衡编程的易用性和优化器设计中保留更多易于被识别出的优化机会，TensorFlow 提供了多套有着不同抽象等级的前端 API 以及计算图上的控制流原语。

为了提高可理解性和编程效率，避免开发者直接操作底层算子，这些计算图中的控制流原语会被封装为前端的控制流 API，图 22.5.2 是用户使用前端基础控制流 API 编写带条件和循环的计算，以及它们所对应的计算图表示。为了简化开发者识别计算图中的控制结构，TensorFlow 基于底层控制流原语，引入高层 Functional 控制流算子，同时添加高层控制流算子向底层控制流算子的转换逻辑。

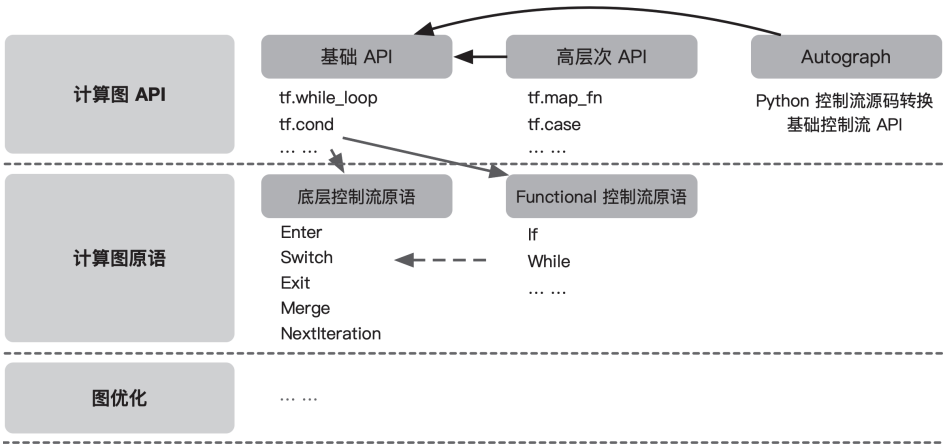


图 22.5.2

下面以循环嵌套两个 for 循环代码，使用 TensorFlow 2.X 的 API 为例子：

```
i = tf.constant(0)
j = tf.constant(0)
a = lambda i: tf.less(i, 10)
b = lambda i: (tf.add(i, 1), )
c = lambda i: (tf.add(j, 1), )
y = tf.while_loop(a, b, [j])
r = tf.while_loop(c, y, [i])
```

TensorFlow 的计算图，每个算子的执行都位于一个执行帧（Execution Frame）中，每个执行帧具有全局唯一的名字作为标识符，控制流原语负责创建和管理这些执行帧。可以将执行帧类比为程序语言中的域（Scope），其中通过 key-value 表保存着执行算子所需的上下文信息，如输入输出变量存储位置等。当计算图中引入控制流后，每个算子有可能被多次执行，控制流原语会在运行时创建这些执行帧，执行帧可以嵌套，对应了开发者写出的嵌套控制流。

```
# 具有全局唯一的名字作为标识符
execution frame: {
  # 保存着执行算子所需的上下文信息
  key(ops_1): value(input_addr, output_addr, ops_attr),
  key(ops_2): value(input_addr, output_addr, ops_attr),
  ...
  # 嵌套 execution frame, 可并发优化
  execution frame: {
    key(ops_1): value(input_addr, output_addr, ops_attr),
    key(ops_2): value(input_addr, output_addr, ops_attr),
    ...
  }
}
```

tf.while_loop 的循环体是一个用户自定义计算子图，对于每个 while 循环，TensorFlow 运行时设置一个执行帧，并在执行帧内运行 while 循环的所有操作。执行帧可以嵌套，嵌套的 while 循环在嵌套的执行帧中运行。嵌套的 tf.while_loop 对应嵌套的计算帧，位于不同计算帧中的算子，只要它们之间不存在数据依赖，就能够被运行时调度并发执行。只要执行帧之间没有数据依赖关系，则来自不同执行帧的操作可以并行运行。

如图 22.5.3 所示，TensorFlow 的原子操作集中有五个控制流原语运算符，其中 Switch 和 Merge 组合起来可以实现条件控制 tf.cond。所有五个基元一起组合则可以实现 while 循环 tf.while_loop。

优点在于：

- 静态图由于能够在执行具体计算之前得到神经网络模型计算的全过程统一描述，使得 AI 编译器在编译优化期间能够利用计算图的信息进行优化。

- 执行逻辑无需在前端宿主语言与运行时之间反复切换，因此往往有着更高的执行效率。

不过这种方式也有比较大的**缺点：**

- 由于控制流原语的语义设计，首要提升运行时的并发数和高效执行，与开发者在描述神经网络模型时使用 Python 语法在编程习惯上差异较大，对开发者来说存在一定的易用性困扰。

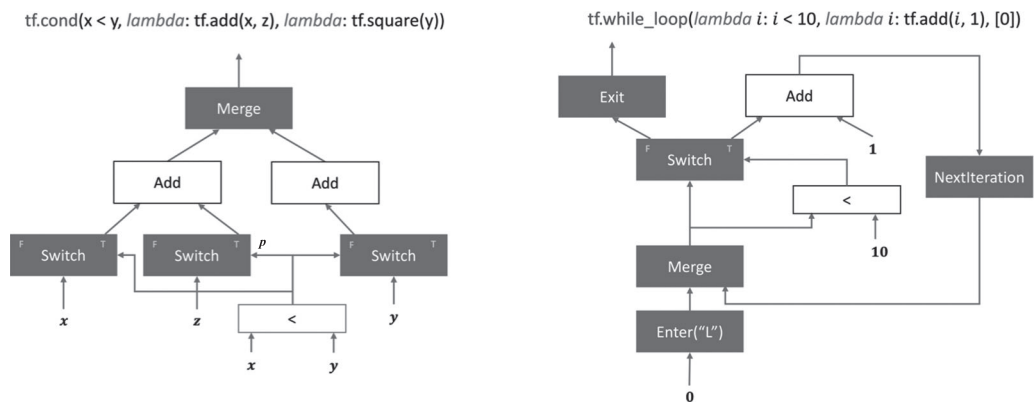


图 22.5.3

● 为了解决让开发者对细节控制原语不感知，需要对控制流原语进行再次封装，以控制流 API 的方式对外提供，这也导致了构建计算图步骤相对复杂。

2. 源码解析

具体实现的过程中，计算图对能够表达的控制流直接展开，如 for 循环内部直接展开成带顺序的多个计算子图。通过创建子图，运行时动态选择子图执行，如遇到 if 和 else 分支时分别创建 2 个子图存放在内存，当控制模块判断需要执行 if 分支的时候把 if 分支的子图产生的序列调度进入执行队列中。

通过对高级语言的源码解析成计算图，再对计算图进行展开的方式，其优点在于：

● 用户能够一定程度自由地使用前端宿主的控制流语言，即在带有约束的前提下使用部分 Python 代码；

- 解耦宿主语言与执行过程，加速运行时执行效率；
- 计算图在编译期得到全计算过程描述，发掘运行时效率提升点；

因为属于静态图的方式，因此继承了声明式编程的缺点：

- 硬件不支持的控制流方式下，执行流仍然会在不同编程语言的边界来回跳转，带来运行时开销；
- 部分宿主的控制流语言不能表示，带有一定约束性。

小结与思考

- 计算图的控制流实现对于构建复杂神经网络结构至关重要，涉及动态图和静态图两种模式，以及复用宿主语言、支持控制流原语和源码解析三种设计思路。
- 动态图方法(如 PyTorch)利用 Python 控制流的易用性高但运行时开销大，而静态图(如 TensorFlow)执行效率高但编程复杂度增加。

- 源码解析方法（如 MindSpore）在易用性和执行效率之间取得平衡，但对高级语言的控制流有一定约束。

22.6 动态图与静态图转换

从 TensorFlow、PyTorch 到 MindSpore，主流的 AI 框架动静态图转换经历了动静分离、动静结合到动静统一的发展过程（图 22.6.1），兼顾动态图易用性和静态图执行性能高效两方面优势，现代 AI 框架具备动态图转静态图的功能，支持使用动态图编写代码，框架自动转换为静态图网络结构执行计算。



图 22.6.1

短短七八年时间，动静态图互相转换的技术在 AI 系统领域发展迅速，大大提升了 AI 算法/模型的开发效率，提高了 AI 产品应用的便利性，实现了计算效率和灵活性的平衡。

22.6.1 背景

前面已详细讲述过动态图与静态图的具体内容，**静态图**易于优化但灵活性低，**动态图**灵活性高但由于缺少统一的计算过程表示，难以在编译期进行分析。

1. 历史发展

1) 动静分离

动静分离阶段，其对应的代表性框架为谷歌在 2017 年发布的 TensorFlow，默认使用静态图，硬件亲和性能高，易部署。相对而言，2018 年发布的 PyTorch，默认采用动态图，其编程范式符合日常编程风格习惯，学习成本低，开发效率高。

2) 动静结合

动态图和静态图都有其各自的优缺点，不过动态图难以转换成为静态图，那么能否让开发者通过标识符号，来通知 AI 框架对于表示的代码段实现静态图转换成为动态图。该思路的基本实现方式是在静态图代码块上加上装饰符。

2019 年谷歌发布的 TensorFlow 2.0 支持动静态图的相互转化，TensorFlow 2.0 默认使用动态

图，允许将一部分 Python 语法转换为可移植、高性能、语言无关的 TensorFlow1.X 语法，从而支持 TensorFlow1.X 静态图和 TensorFlow2.X 动态图的切换。2022 年 Meta 发布的 PyTorch 2.0 从 Python 侧支持 Compiler 编译产生静态图的功能。

3) 动静统一

动静统一是 AI 框架技术上追求的最终实现方式，开发者能够根据需要，灵活地在动态图与静态图之间切换。然而动态图和静态图的切换面临很大困难：静态图可以看作是一种特殊的领域语言（DSL），则该特殊 DSL 实际上是表示为静态语言的；但是 Python 实际为动态解释类型语言，因此很难将 Python 语言无损转化到静态语言中，又可以方便地从静态语言转换回 Python 代码。

2. 实现方式

主流的 AI 框架最终目标是实现计算图的**动静统一**，目前从 AI 框架的技术趋势来看，动态图与静态图的融合在不断向前探索过程中：前端用户使用宿主语言（如 Python）中的控制流语句编写神经网络模型，调试完后，由 AI 框架自动转换为静态图的结构。而动态图向静态图的转换分为基于追踪（Tracing）和基于源代码解析（Parsing）两种方式：

- **基于追踪**：直接执行开发者编写的 Python 代码，记录下算子调用序列，将算子调用序列保存为静态图，执行中脱离前端高级语言环境，由运行时 Runtime 按照静态图逻辑执行。即动态图模式执行并记录调度的算子，构建和保存为静态图模型。

- **基于源代码解析**：以高级语言的抽象语法树（AST）作为输入，通过 AI 框架定义的计算图 IR 转化为框架内部的语法树，经过别名分析、SSA（Static Single Assignment）、类型推断等编译器中间件 Pass，最终转换为静态计算图表示。即分析前端源代码将动态图转为静态图，并在框架层帮开发者使用静态图执行器运行。

不过在具体实现方式下，解决动态图和静态图转换的问题时，主要有以下两条路径：

- **动态转静态**：从动态图出发，AI 框架可以在运行过程中自动通过 JIT，无需用户用修饰符指定，如 PyTorch 的 Lazy Tensor 和 Compiler；

- **静态转静态**：从静态图出发，编译过程中如果发现有不支持的语法，保留到运行时进行 Fallback 回 Python，如 PyTorch 框架中的 JIT Fallback。

22.6.2 追踪模式

基于追踪的方式会直接执行用户代码，记录下算子调用序列，将算子调用序列保存为静态图模型，在后续编译、运行时的执行过程中脱离前端高级编程语言 Python 环境，最后交由框架后端运行时按照静态图调度与执行。

动态图基于前端宿主语言的解释器对网络模型的代码进行解析执行。因此，基于追踪的动态静态图转换的原理相对简单，当使用动态图模式构建好网络模型后，使用追踪的方式进行转换将分为两个阶段：

- 第一阶段：与动态图生成原理相同，AI 框架创建并运行动态图代码，自动追踪计算图中数据流的流动以及算子的调度，将所有的算子捕获并根据调度顺序构建静态图模型。与动态生成不同的地方在于 AI 框架并不会销毁构建好的计算图，而是将其保存为静态图留待后续执行计算。

- 第二阶段：当执行完一次动态图后，AI 框架已生成静态图，当再次调用相同的模型时，AI 框架会自动指向静态图模型执行计算。追踪技术只是记录第一次执行动态图时调度的算子，但若是模型中存在依赖于中间结果的条件分支控制流，只能追踪到根据第一次执行时触发的分支。

基于追踪模式构建的静态图模型并不是完整的计算图，缺失了数据未流向的其他分支。在后续的调用中，因为静态模型已经生成无法再次改变，除非重新生成计算图，若计算过程中数据流向缺失分支会导致模型运行错误。同样，依赖于中间数据结果的循环控制也无法追踪到全部的迭代状态。

```
w_h = torch.randn(20, 20, requires_grad=True)
w_x = torch.randn(20, 10, requires_grad=True)
x = torch.randn(1, 10)
prev_h = torch.randn(1, 20)

h2h = torch.mm(W_h, prev_h.t())
if h2h > w_x:
    i2h = torch.mm(w_x, x.t())
    h2h = h2h + i2h
next_h = h2h.tanh()

loss = next_h.sum()
loss.backward()
```

基于追踪模式的**难点**在于：通过 Tracing 的方式获取的计算图，实际上不是一个有向无环图（DAG），而是一个平铺算子执行流，所以很难处理控制流。比如循环 while、Loop、for，对于 Tracing 的方式来说就是展开循环体，但是有些情况下循环体无法有效展开，如循环条件根据训练的收敛情况/算子的执行结果而改变等。因此上面的代码产生的计算图有 2 种可能性，如图 22.6.2 所示。

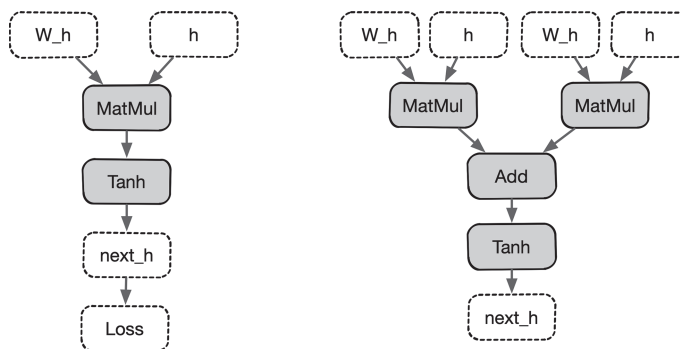


图 22.6.2

基于追踪模式的特点总结如下：

- **优点：**简单易于实现；能够更为广泛地支持前端宿主语言中的各种动态控制流语句，例如：函数调用、函数嵌套、函数递归等。
- **缺点：**执行场景受限，Tracing 直接执行程序一次，只能保留程序有限执行轨迹并线性化，静态图失去源程序完整控制结构。

22.6.3 源码解析

静态图模式下需要经过 AI 框架自带的图编译器对模型进行建图后，再执行静态计算图。由于图编译器所支持编译的静态图代码与动态图代码之间存在差异，因此基于源码转换的方式，需要将动态图代码转换为静态图代码描述，最后经过图编译器生成静态计算图。

基于源代码解析的方式则能够改善基于追踪转换的缺陷，其流程经历三个阶段：

- **第一阶段：**以宿主语言的抽象语法树（Abstract Syntax Tree, AST）为输入；对动态图模式下的宿主语言代码扫描进行词法分析，通过词法分析器分析源代码中的所有字符，对代码进行分割并移除空白符、注释等，将所有的单词或字符都转化成符合规范的词法单元列表。这一阶段，需要严格地筛选前端宿主语言语法要素，往往只会解析宿主语言中的一个子集，而非前端宿主语言所有特性表达都能解析。
- **第二阶段：**以词法分析器的结果作为输入，接着进行语法分析（即 AI 框架编译层的解析器），将得到的词法单元列表转换成语法树的形式，并对语法进行检查，避免错误。接着将宿主语言的抽象语法树，整理成一个 AI 框架内部的抽象语法树表示。
- **第三阶段：**从 AI 框架的内部语法树开始经过别名分析、SSA 化、类型推断等重要分析，最终转换为静态计算图表示。动态图转静态图的核心部分就是对抽象语法树进行转写，AI 框架中对每一个需要转换的语法都预设转换器，每一个转换器对语法树进行扫描改写，将动态图代码语法映射为静态图代码语法。

上面流程中，最为重要的前端宿主语言的控制流，会在第三阶段分析转换为静态图接口进行实现，能够有效避免基于追踪的方式中控制流缺失的情况。计算图转换之后，可以从新的语法树或者计算图还原出可执行的静态图代码。

```
@torch.jit.script
def fool(x: Tensor, y: Tensor, z: Tensor) -> Tensor:
    if x < y:
        s = x + y
    else:
        s = torch.square(y)
    return s

@torch.jit.script
def foo2(s: Tensor) -> Tensor:
    for i in torch.range(10):
        s += i
    return s
```


上面的代码片段是使用了 PyTorch 的 Script 模式（基于源代码解析），将动态图转换为静态图执行，图 22.6.3 是 PyTorch 背后的处理过程。

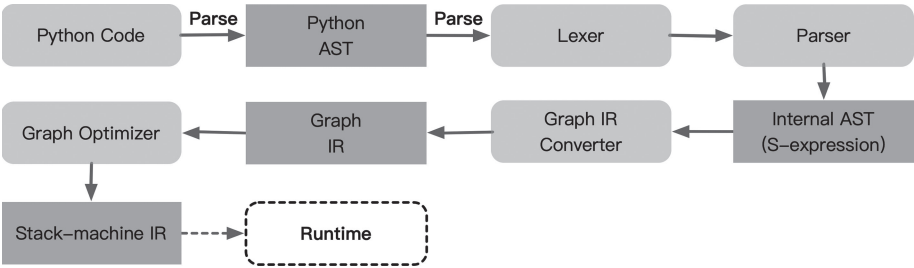


图 22.6.3

基于源代码解析的**难点**在于：AI 框架是从前端宿主语言 Python 进行 AST 转换而来，好处是控制流和神经网络模型的属性信息都可以保留下来，但是挑战是 Python 的大部分语法和数据结构都要转换为静态图的表达，更难的是 Python 是动态类型语言，所以从 AST 到静态图的转换中需要一个复杂的类型/值推导过程，导致实现困难与复杂。

PyTorch Dynamo 特性属于基于源码转换,不过转换层不再是对 Python 的语言进行转换,而是在 Python 的解释器中转换成自定义的 FX Graph,从而更好地复用宿主语言的高级特性。

基于源代码解析方式的特点总结如下：

- **优点：**能够更广泛地支持宿主语言中的各种动态控制流语句。
- **缺点：**后端实现和硬件实现会对静态图表示进行限制和约束，多硬件需要切分多后端执行逻辑；宿主语言的控制流语句并不总是能成功映射到后端运行时系统的静态图表示；遇到过度灵活的动态控制流语句，运行时会退回 Fallback 由前端语言跨语言调用驱动后端执行。

小结与思考

- 动态图与静态图的转换技术发展迅速，AI 框架通过动静结合的方式，实现了动态图的易用性与静态图的高执行性能的平衡。
- 动态图到静态图的转换主要基于追踪（Tracing）和源代码解析（Parsing）两种方式，Tracing 通过记录算子调用序列构建静态图，而源代码解析通过 AST 转换为静态图表示。
- 动静转换技术面临的挑战包括控制流的准确表示、类型推断的复杂性以及硬件和后端实现的限制，但这些技术的发展极大地提升了 AI 算法的开发效率和产品的便利性。